Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

JAVA ASSIGNMENT

NAME:Kavya Gada

SAP ID:60017230102

ROLL NO.:A018

BRANCH:AIML

BATCH:A1

DATE OF SUBMISSION:06/04/24

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

[Type here]

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Kavya Gada

Kavya Gada
AI-ML
AOIS
A-1

Pg-1

Exp-8

Aim:- To implement Inheritance and super keyword.

Description:-

(i) Types of inheritances:-

⇒ • Single inheritance: Here, a base/sub class is derived from only one super class. The properties inherited are of that single parent class.

⇒ • Multi-level inheritance: Here, a class is derived from a class which was already derived from a previous class. This means that there are multiple intermediate base classes between the main parent/super class and main derived class

⇒ Hierarchical inheritance: In this case, many classes are derived from a single base class.

⇒ Hybrid inheritance: This inheritance is a combination of 2 or more types of above mentioned inheritance

ii) Syntax to create subclass/Superclass relationship

The Keyword used to create subclass/superclass relationship is 'extends'.
The Syntax is

P.T.O.

FOR EDUCATIONAL USE

[Type here]

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

2

```
class Superclass
{
// methods, constructors, etc
}
class SubClass extends SuperClass
{
// methods, constructors, etc
}
```

(iii) Access modifier and visibility in java. :-

Four main access modifier are:

- Public: Members of 'public' modifier are accessible from any other class.
- Protected: These members are accessible within the same package and by subclasses (even if they are in a different package)
- Private: The members under this modifier are accessible only within the same class, not even in the sub-class.
- Default: members with no explicit access modifier are ~~accessible~~ accessible in a subclass.

→ Public modifier members have the widest visibility.

(v) Role of Constructors in inheritance :-

FOR EDUCATIONAL USE

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

3

- Constructors ensure intialization of the superclass and subclass e in inheritance.
- When a derived class is extended from the base class, the constructor of the base class is ~~ex~~ executed first followed by the constructor of the derived class.

(v) Super Keyword used variables, constructors and methods :-

- In variable, 'super' can be used to access superclass variable and distinguish it with same name variable in subclass.
- For constructor, 'super' is used to explicitly call a superclass constructor from a subclass.
- 'super' can also be used to call superclass methods from a subclass method.

NAME:Kavya Gada

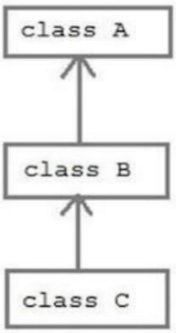
SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Experiment No: 8

Q1] WAP to demonstrate the role of Constructors in inheritance in the following class diagram.



Code:

```
class A {
public A() {
System.out.println("Constructor of class A");
}
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
class B extends A {
public B() {

System.out.println("Constructor of class B");
}
}
class C extends B {
public C() {
System.out.println("Constructor of class C");
}
}
public class exp8 {
public static void main(String[] args) {

C objC = new C();
}
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Output :

```
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac exp8.java

C:\java program>java exp8
Constructor of class A
Constructor of class B
Constructor of class C

C:\java program>_
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Q2] WAP to create a superclass having a variable. Let the variable be initialised to some value within a constructor. This class should have a method display () to display the initial value of the variable. Derive a subclass that accesses the constructor, variable and method of the super class using super keyword.

CODE:

```
class SuperClass {
int variable;
public SuperClass(int value) {
this.variable = value;
}
public void display() {
System.out.println("Initial value of variable: " + variable);
}
}
class SubClass extends SuperClass {
public SubClass(int value) {
super(value); }
public void displayInitialValue() {
super.display();
}
}
public class exp81 {
public static void main(String[] args) {
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
SubClass obj = new SubClass(10);
obj.displayInitialValue();
}
}
```

OUTPUT:

```
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac exp81.java

C:\java program>java exp81
Initial value of variable: 10

C:\java program>_
```

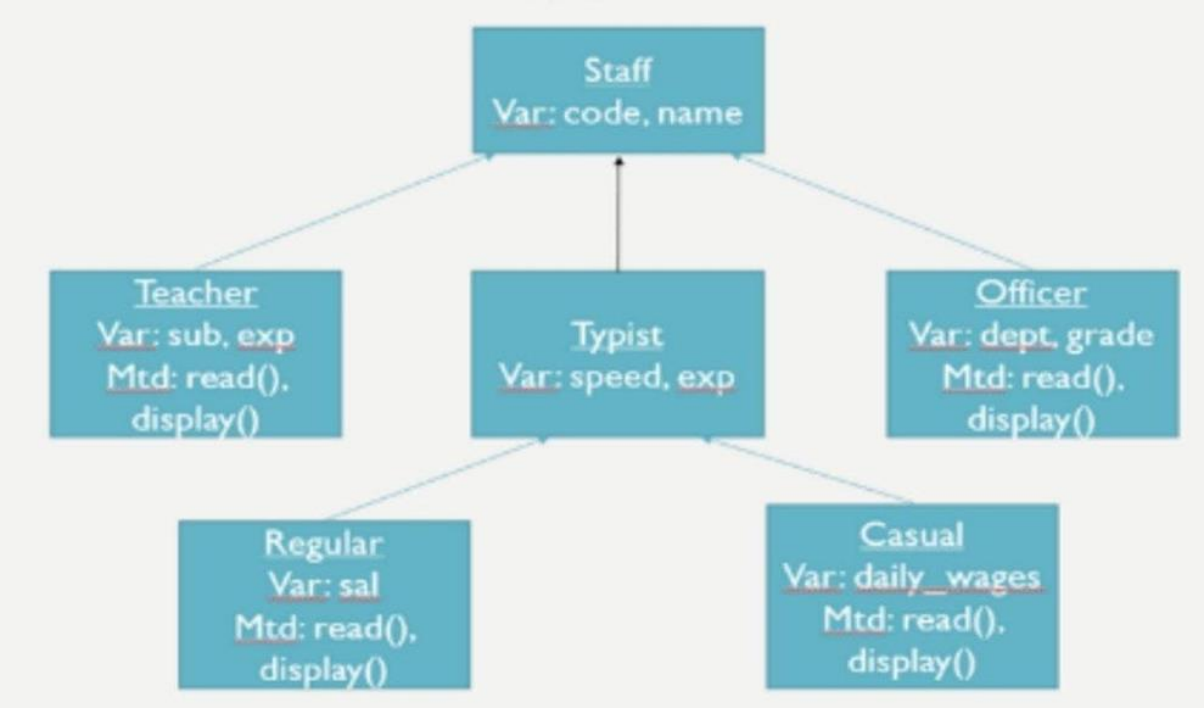

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Q3] Display data of the specialized classes given in the following class diagram.



CODE:

```
import java.util.*;
class Staff{
int code;
String name;
Scanner input1 = new Scanner(System.in);
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
Scanner input2 = new Scanner(System.in);
}
class Teacher extends Staff{
String sub;
int exp;
void read(){
System.out.println("For Teachers:-");
System.out.print("Enter Name: ");
name = input1.next();
System.out.print("Enter Code: ");


code = input2.nextInt();

System.out.print("Enter Subject: ");
sub = input1.next();
System.out.print("Enter Experience: ");

exp = input2.nextInt();
}
void display(){
System.out.println("\nTeaching Staff Info:-");
System.out.println("Name \t Code \t Subject \t Experience");
System.out.println(name+" \t "+code+" \t "+sub+" \t\t "+exp);
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
}
class Officer extends Staff{
String grade, dept;
void read(){
System.out.println("\nFor Officers:-");
System.out.print("Enter Name: ");
name = input1.next();
System.out.print("Enter Code: ");
code = input2.nextInt();
System.out.print("Enter Grade: ");
grade = input1.next();
System.out.print("Enter Department: ");

dept = input1.next();
}
void display(){
System.out.println("\nOfficer Staff Info:-");
System.out.println("Name \t Code \t Grade \t Department");
System.out.println(name+" \t "+code+" \t "+grade+" \t\t "+dept);
}
}
class Typist extends Staff{
int speed, exp;
}
class Regular extends Typist{
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
int salary;
void read(){
System.out.println("\nFor Regular Typist:-");
System.out.print("Enter name: ");
name = input1.next();
System.out.print("Enter Code: ");
code = input2.nextInt();
System.out.print("Enter Speed: ");
speed = input1.nextInt();
System.out.print("Enter Expirence: ");
exp = input2.nextInt();
System.out.print("Enter Salary: ");
salary = input2.nextInt();

}
void display(){
System.out.println("\nRegular Typist Staff Info:-");
System.out.println("Name \t Code \t Speed \t Experience \t
Salary");
System.out.println(name+" \t "+code+" \t "+speed+" \t\t "+exp+"
\t"+salary);
}
}
class Casual extends Typist{
int dailyWages;
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
void read(){
System.out.println("\nFor Casual Typist:-");
System.out.print("Enter name: ");
name = input1.nextLine();
System.out.print("Enter Code: ");
NAME: SAUMYA NIMESH SHAH ROLL NO:A044 SAP ID:60017230090
code = input2.nextInt();
System.out.print("Enter Speed: ");
speed = input1.nextInt();
System.out.print("Enter Experience: ");
exp = input2.nextInt();
System.out.print("Enter Daily Wages: ");
dailyWages = input2.nextInt();
}
void display(){
System.out.println("\nCasual Typist Staff Info:-");
System.out.println("Name \t Code \t Speed \t Experience \tDaily
Wages");
System.out.println(name+" \t "+code+" \t "+speed+" \t\t "+exp+"
\t\t"+dailyWages);
}
}
public class exp8{
public static void main(String[] args){
Teacher t = new Teacher();
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
Officer o = new Officer();
Regular r = new Regular();
Casual c = new Casual();
t.read();
t.display();
o.read();
o.display();
r.read();
r.display();
c.read();
c.display();
}
}
```

OUTPUT:

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

```
C:\java program>java exp8
For Teachers:-
Enter Name: rahul
Enter Code: 001
Enter Subject: maths
Enter Experience: 5

Teaching Staff Info:-
Name      Code     Subject          Experience
rahul     1        maths            5

For Officers:-
Enter Name: xyz
Enter Code: 002
Enter Grade: x
Enter Department: child safety

Officer Staff Info:-
Name      Code     Grade    Department
xyz       2        x                 child

For Regular Typist:-
Enter name: abc
Enter Code: 003
Enter Speed: 100
Enter Expirence: 6
Enter Salary: 30000

Regular Typist Staff Info:-
Name      Code     Speed    Experience       Salary
abc       3        100              6       30000

For Casual Typist:-
Enter name: cvb
Enter Code: 004
Enter Speed: 80
Enter Experience: 4
Enter Daily Wages: 2500

Casual Typist Staff Info:-
Name      Code     Speed    Experience      Daily Wages
cvb       4        80               4               2500

C:\java program>
```

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

CONCLUSION:

In summary, understanding inheritance and its various forms, along with the syntax for creating subclass/superclass relationships, access modifiers, and the role of constructors and the super keyword, is essential for effective object-oriented programming in Java. These concepts form the backbone of Java's object-oriented paradigm, facilitating code reuse, modularity, and extensibility.

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018